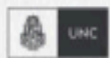


CERTIFICACIÓN UNIVERSITARIA



FCEFYN



Manual para el
alumno

Build & Package
Manager Tools



**WE WANT
SKILLS!**

mundosE

Manual para el alumno: Build & Package Manager Tools.....	2
Objetivo del encuentro:.....	2
Contenido a desarrollar:.....	2
1. Qué es build y por qué es la base del empaquetado.....	3
2. Gestores de paquetes y motores de build por ecosistema.....	4
3. Versionado semántico, rangos, lockfiles y políticas de actualización.....	6
4. SBOM, licencias y trazabilidad: por qué importan.....	7
5. Firmas, SLSA y repositorios: elevar la confianza end-to-end.....	8
Bibliografía.....	10

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este se presenta un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el alumno: Build & Package Manager Tools

Objetivo del encuentro:

Comprender, de punta a punta, el flujo de Build & Package Manager en DevOps para producir artefactos trazables, reproducibles y seguros, y publicarlos/consumirlos de manera controlada dentro de un pipeline de CI/CD.

Contenido a desarrollar:

- ✓ Distinguir con claridad *build*, empaquetado y publicación.
- ✓ Comprender cómo usar SemVer, manejar dependencias transitivas y lockfiles para asegurar reproducibilidad.
- ✓ Reconocer el rol de los gestores de paquetes y de los repositorios de artefactos (proxy, caché, control y auditoría).
- ✓ Incorporar prácticas de seguridad de la cadena de suministro: SBOM, firmas y niveles SLSA.
- ✓ Aplicar la idea de promoción sin rebuild para mantener identidad e integridad del artefacto entre entornos.
- ✓ Definir criterios mínimos de gobernanza (licencias permitidas, umbral de CVEs y excepciones con vencimiento) que después vas a usar en el laboratorio.

1. Qué es *build* y por qué es la base del empaquetado

En DevOps, **build** es el proceso que transforma tu código fuente y sus dependencias en un **artefacto** listo para ser distribuido o ejecutado. No se trata solo de “compilar”: un build moderno ejecuta pruebas, calcula **hashes** de integridad, agrega **metadatos** (nombre, versión, autoría, procedencia), y produce salidas **inmutables** que pueden ser auditadas y promovidas entre entornos. Este enfoque ayuda a responder preguntas clave: ¿de qué commit salió este binario?, ¿lo puedo recrear bit a bit?, ¿quién lo generó y bajo qué condiciones? Cuando estas respuestas son claras, la organización gana **confianza** y reduce el riesgo de errores difíciles de reproducir.

La **trazabilidad** es el primer beneficio tangible. Asociar un artefacto con un commit, una rama y una corrida de CI/CD permite reconstruir la historia de cómo se generó. La **reproducibilidad** (o determinismo) es el segundo: con las mismas entradas (fuentes, versiones de dependencias, configuraciones) deberías obtener **exactamente** el mismo resultado. Para lograrlo, necesitas controlar fuentes de no determinismo: marcas de tiempo, ordenamiento de archivos, generación de código, y hasta las versiones de las herramientas del compilador. Este control se complementa con **lockfiles** (archivos que congelan la resolución exacta del grafo de dependencias) y con **caches** bien diseñadas (de dependencias y de tareas), que aceleren el proceso y eviten sorpresas entre máquinas.

El build prepara el terreno para el **empaquetado**, que es la etapa donde describes formalmente lo que construiste: nombre del paquete, **versión** (generalmente bajo **SemVer**, ver más abajo), compatibilidad, **licencias**, dependencias directas, *scripts* o tareas pre/post, y cualquier otra información relevante. Toda esa “documentación ejecutable” viaja con el artefacto. Cuando el empaquetado está bien hecho, otro proyecto puede **consumir** tu paquete con previsibilidad, y un equipo de auditoría puede revisar **qué contiene** y **qué licencia** cubre cada pieza.

Por último, el artefacto se **publica** en un **repositorio** que impone políticas: quién puede subir, quién puede descargar, qué **formatos** están permitidos y por cuánto tiempo se retienen. Publicar no es simplemente “subir a un server”: es el punto donde **seguridad** y **compliance** se vuelven visibles. Ahí se anexan **SBOM** (inventario de componentes), **firmas** digitales del artefacto y **checksums**. Todo junto conforma la **evidencia** que necesitas si aparece una vulnerabilidad (por ejemplo, una **CVE**) y hay que responder rápido: ¿estamos afectados?, ¿en qué versiones?, ¿en qué entornos?

Si te quedas con una idea, que sea esta: *build* es el **piso** de todo lo demás. Si el build es reproducible y deja buena evidencia (metadatos, hashes, logs), empaquetar, firmar, publicar y consumir se vuelve un proceso **confiable**. En el

laboratorio vas a ver cómo estos conceptos aterrizan en tareas concretas: generar un SBOM, firmar una imagen **OCI** y publicarla en un repositorio interno para luego **promoverla** entre entornos sin recompilar.

Glosario mínimo de este punto:

Artefacto: producto resultante del build (binario, librería, imagen, etc.).

Hash: huella criptográfica del artefacto para verificar integridad.

Lockfile: archivo que fija la resolución exacta de dependencias.

CVE: identificador público de vulnerabilidades conocidas.

Recursos para profundizar:

- Semantic Versioning 2.0.0: <https://semver.org/>
- Reproducible Builds (sitio oficial): <https://reproducible-builds.org/>
- @vsoch © (2019) OCI Image Specification (Open Container Initiative): <https://specs.opencontainers.org/image-spec/>

2. Gestores de paquetes y motores de build por ecosistema

Cada ecosistema de lenguaje trae herramientas para declarar dependencias, automatizar tareas y empaquetar. En Java/Android, Maven organiza la construcción con un modelo declarativo (POM) y fases predefinidas (compilar, testear, empaquetar, instalar, desplegar). Gradle aporta flexibilidad con un DSL que permite definir tareas y depende de un potente caché de tareas para acelerar. ¿Cuál conviene? Depende de tu equipo: si priorizan convención y predictibilidad, Maven; si necesitan personalización y performance, Gradle. En JavaScript/TypeScript, npm es el gestor por defecto; Yarn se consolidó ofreciendo instalaciones determinísticas y pnpm optimiza espacio usando almacenamiento direccionado por contenido y una topología estricta de node_modules.

En Python, pip es el instalador tradicional, pero Poetry resolvió necesidades modernas: declarar dependencias en pyproject.toml, generar lockfiles y publicar paquetes en índices. La Python Packaging User Guide explica cuándo conviene usar entornos virtuales, cómo fijar versiones y qué buenas prácticas aplicar para evitar conflictos entre proyectos. En .NET, NuGet es el mecanismo estándar para crear y consumir paquetes; integra muy bien con los IDEs y con pipelines. Rust brilla con Cargo, que unifica dependencias, build y publicación; Go utiliza Go Modules con un enfoque descentralizado. En C/C++, CMake genera proyectos para diversas toolchains e IDEs, y Make ejecuta recetas; su combinación sigue siendo predominante en proyectos de bajo nivel.

Fuera del mundo “bibliotecas”, el empaquetado de aplicaciones como contenedores usa el estándar OCI. Una imagen OCI es un conjunto de capas + un manifiesto + una configuración. Este formato habilita que cualquier herramienta compatible pueda construir, firmar, escanear y ejecutar la misma imagen. El resultado: menos fricción y más interoperabilidad.

La moraleja transversal es que todas estas herramientas, más allá de sus diferencias, apuntan al mismo objetivo: producir artefactos trazables y gobernados. Lo que cambia es el “contrato” de cada ecosistema (manifiestos, lockfiles, convenciones) y su integración con los repositorios internos y el pipeline de CI/CD. En el laboratorio vas a comparar estas ideas mirando qué campos tiene un manifiesto, cómo se refleja la versión en el artefacto y de qué manera se publica en un repositorio.

Glosario mínimo de este punto:

POM: archivo de Maven que describe el proyecto y sus dependencias.

DSL: lenguaje específico de dominio para describir tareas de build.

OCI: estándar de contenedores que define imágenes interoperables.

Recursos para profundizar:

- **Java/Android:**
 - The Apache Software Foundation © (2002–2025). Maven Guides: <https://maven.apache.org/guides/index.html>,
 - Gradle, Inc. (2025) Gradle User Manual: <https://docs.gradle.org/current/userguide/userguide.html>
- **JS/TS:**
 - npm Docs: <https://docs.npmjs.com/>
 - Yarn Docs: <https://yarnpkg.com/>
 - pnpm Docs: <https://pnpm.io/>
- **Python:**
 - PyPA © (2025). Python Packaging User Guide: <https://packaging.python.org/>
 - The pip developers pip © (2008-2020). Docs: <https://pip.pypa.io/>
 - Vercel (2025). Poetry Docs: <https://python-poetry.org/docs/>
- **.NET:** Microsoft © (2025). NuGet (Microsoft Learn): <https://learn.microsoft.com/en-us/nuget/>
- **Rust / Go:**
 - The Cargo Book: <https://doc.rust-lang.org/cargo/>
 - Go Modules Reference: <https://go.dev/ref/mod>
- **C/C++:**
 - Kitware, Inc. (2025). CMake Reference: <https://cmake.org/cmake/help/latest/>
 - GNU Make Manual: <https://www.gnu.org/software/make/manual/>
- **Contenedores:** @vsoch © (2019) OCI Image Spec: <https://specs.opencontainers.org/image-spec/>

3. Versionado semántico, rangos, lockfiles y políticas de actualización

El **versionado semántico (SemVer)** es una convención ampliamente adoptada para comunicar el **impacto** de un cambio. Representa versiones como **Mayor.Menor.Patch**: si el número **Mayor** cambia (por ejemplo, de 1.x a 2.0.0), la expectativa es que existan **cambios incompatibles**; si cambia **Menor** (1.4.2 a 1.5.0), deberían sumarse **funciones nuevas** sin romper compatibilidad; un cambio en **Patch** (1.4.2 a 1.4.3) indica **correcciones** y mejoras menores. Entender SemVer te ayuda a decidir **cuándo** actualizar y **con qué cautelas**.

Sin embargo, el número de versión no vive solo: casi todos los proyectos dependen de otros. El **grafo de dependencias** (directas y **transitivas**) puede cambiar sin que toques nada, si dejas rangos “abiertos”. Para evitar sorpresas, los ecosistemas incluyen **lockfiles** (o mecanismos equivalentes) que **congelan** la resolución exacta de ese grafo. Con un lockfile, lo que probaste en tu máquina será lo mismo que corra en CI y lo mismo que llegue a producción. Esto contribuye a la **reproducibilidad** y reduce los “bugs fantasma”.

En cuanto a **políticas**, es sano definir:

- (a) qué **rangos** de versión aceptas automáticamente (solo *patch*, *minor + patch*, etc.);
- (b) **ventanas de actualización** (semanales, quincenales) con pruebas de regresión;
- (c) cómo reaccionas ante una **CVE** crítica (excepción controlada); y,
- (d) reglas para **licencias** aceptables.

Estas políticas se documentan y se hacen cumplir en el pipeline con **gates** automáticos (por ejemplo, fallar si una dependencia trae una licencia no permitida o una vulnerabilidad severa sin mitigación).

Por último, la estrategia de **promoción sin rebuild** completa la historia: construir **una vez, firmar, publicar** y mover **el mismo artefacto** por dev/test/prod. Así, los hashes y firmas siguen siendo válidos y evitas que sutiles diferencias entre compilers o tiempos modifiquen el resultado. Desde el punto de vista de operación, esta práctica simplifica auditorías y acelera *rollbacks*: si algo falla, se puede retroceder al artefacto anterior con total certeza de **identidad**.

Glosario mínimo de este punto:

SemVer: convención **Mayor.Menor.Patch** para comunicar impacto.

Rango de versiones: especificación que permite aceptar versiones dentro de ciertos límites.

Promoción: mover el mismo artefacto entre entornos sin recompilar.

Recursos para profundizar:

- Semantic Versioning 2.0.0: <https://semver.org/>
- Gradle, Inc. (2025). Gradle Platforms (alineación de dependencias): <https://docs.gradle.org/current/userguide/platforms.html>
- PyPA © (2025). Python Packaging User Guide — Installing packages: <https://packaging.python.org/tutorials/installing-packages/>

4. SBOM, licencias y trazabilidad: por qué importan

Un **SBOM** (*Software Bill of Materials*) es un **inventario formal** de todo lo que compone un artefacto: componentes, versiones, orígenes, **licencias** y relaciones. Piénsalo como la lista de ingredientes en un alimento: cuando aparece una alerta sanitaria, sabes qué lotes están afectados; con un SBOM, si surge una **CVE** en una librería, puedes localizar **qué versiones** de tu software la incluyen y **dónde** están desplegadas. Por eso, cada vez más organizaciones y reguladores exigen **SBOMs**.

Existen estándares que facilitan la **interoperabilidad**: **SPDX** (de la Linux Foundation, también estándar ISO/IEC) y **CycloneDX** (de OWASP). Ambos describen de manera estructurada los componentes y sus atributos (incluidas licencias mediante la **SPDX License List**). Esto permite que herramientas automáticas crucen el SBOM con bases de vulnerabilidades y políticas de licencias. En la práctica, generar un SBOM en el **build** y publicarlo junto al artefacto resuelve dos problemas: (a) **detección rápida** cuando aparece una vulnerabilidad, y (b) **cumplimiento** de licencias (evitar usar algo incompatible con la política interna).

La **trazabilidad** mejora cuando además agregas **procedencia** (*provenance*): un documento/atestación que indica **quién** construyó el artefacto, **cuándo**, con **qué insumos** y **qué receta** (pipeline, parámetros, hashes de entrada). Si esa atestación está **firmada** y respaldada por un registro de **transparencia**, la confianza sube aún más. En la práctica, esto se logra con herramientas que integran **firmas** (p. ej., **cosign**) y que pueden operar en modo **keyless**, usando identidades **OIDC** del pipeline. Así, al verificar, se valida no solo que el archivo **no fue alterado**, sino que proviene del **pipeline oficial** de la organización.

Finalmente, el SBOM también es útil para **gobernanza**: puedes establecer **listas permitidas** de licencias (MIT, Apache-2.0, etc.) y bloquear artefactos que traigan licencias no compatibles. En equipos grandes, centralizar los SBOMs de cada release arma un “**catálogo**” que facilita auditorías y respuestas a incidentes. En el laboratorio, vas a producir un SBOM, asociarlo a tu artefacto e imaginar un escenario: aparece una CVE; ¿cómo la rastreas con tu SBOM?, ¿qué excepciones

documentarías, con **fecha de caducidad**, hasta reemplazar la dependencia?

Glosario mínimo de este punto:

SBOM: inventario de componentes, versiones, licencias y relaciones.

SPDX/CycloneDX: estándares para describir SBOMs.

Provenance: atestación de procedencia del artefacto.

Recursos para profundizar:

- SPDX — sitio oficial: <https://spdx.dev/>
- SPDX License List: <https://spdx.org/licenses/>
- CycloneDX — sitio oficial: <https://cyclonedx.org/>
- The OWASP Foundation. (2024). OWASP CycloneDX — Authoritative Guide to SBOM (PDF):
https://cyclonedx.org/guides/OWASP_CycloneDX-Authoritative-Guide-to-SBOM-en.pdf

5. Firmas, SLSA y repositorios: elevar la confianza end-to-end

Publicar un artefacto sin **pruebas de integridad y autoría** es dejar una puerta entreabierta. Por eso, además de hashes, se usan **firmas** y **registros de transparencia**. Con **Sigstore/cosign**, puedes firmar artefactos (incluidas imágenes **OCI**) y registrar la firma en un libro público auditable. El modo **keyless** —apoyado en **OIDC**— evita manejar llaves de larga vida y facilita la adopción. En la verificación, puedes exigir que “solo se acepten artefactos firmados por el pipeline oficial”, y que además presenten **SBOM** y **atestaciones** válidas. Esto monta un **cerco** que complica ataques por sustitución o contaminación de la cadena de suministro.

El marco **SLSA** ordena esta conversación en **niveles**: a mayor nivel, mayores garantías de integridad/procedencia. No se trata de coleccionar “sellos”, sino de construir una **hoja de ruta** realista: aislar el servicio de build, registrar insumos, firmar outputs, validar automáticamente, y generar evidencia que pueda ser auditada. En organizaciones con múltiples equipos, esta estrategia permite priorizar los pipelines que producen artefactos críticos y avanzar por etapas.

Por su parte, los **repositorios de artefactos**, lugar donde se hace efectiva la política: **Nexus**, **Artifactory** o **GitHub Packages**, permiten segmentar repos **locales** (internos), **remotos** (proxy de terceros) y **virtuales** (vistas unificadas), establecer **controles de acceso**, aplicar **retención/purga**, y adjuntar **metadatos** (SBOM, firmas, checksums). Una **práctica saludable** es promover artefactos **sin recompilar** y etiquetar estados (snapshot, release, aprobado para preproducción, etc.). Si algo falla, la combinación “**misma imagen + mismas firmas + mismo**

SBOM” facilita el *rollback* y la investigación.

En el laboratorio vas a simular este flujo: construir un artefacto, generar su SBOM, firmarlo, publicarlo y luego verificarlo. Ese ejercicio te entrena en la **disciplina** de producir evidencia desde el principio y no “a último momento”. En contextos regulados o de alta criticidad, esta disciplina deja de ser una opción para convertirse en un **requisito**.

Glosario mínimo de este punto:

Firma: comprobación criptográfica de autoría/integridad.

Keyless: firma sin llaves de larga vida, basada en identidad OIDC.

Registro de transparencia: libro público de eventos de firma/verificación.

Recursos para profundizar:

- Sigstore/cosign — Overview de firma *keyless*:
<https://docs.sigstore.dev/cosign/signing/overview/>
- Cosign — documentación: <https://docs.sigstore.dev/cosign/>
- SLSA — sitio oficial: <https://slsa.dev/>
- GitHub, Inc. © (2025). GitHub Packages — documentación:
<https://docs.github.com/packages>
- Sonatype Nexus Repository — documentación:
<https://help.sonatype.com/en/sonatype-nexus-repository.html>
- JFrog Artifactory — documentación:
<https://jfrog.com/help/r/jfrog-artifactory-documentation>

Bibliografía

- Apache Software Foundation. (s. f.). *Maven — Guides*. Recuperado de <https://maven.apache.org/guides/index.html>
- CycloneDX Project (OWASP). (2023). *OWASP CycloneDX — Authoritative Guide to SBOM*. Recuperado de https://cyclonedx.org/guides/OWASP_CycloneDX-Authoritative-Guide-to-SBOM-en.pdf
- CycloneDX Project (OWASP). (s. f.). *CycloneDX — Sitio oficial*. Recuperado de <https://cyclonedx.org/>
- Free Software Foundation. (2023). *GNU Make Manual*. Recuperado de <https://www.gnu.org/software/make/manual/>
- Go Team. (s. f.). *Go Modules Reference*. Recuperado de <https://go.dev/ref/mod>
- Gradle Inc. (s. f.). *Gradle User Manual*. Recuperado de <https://docs.gradle.org/current/userguide/userguide.html>
- Kitware. (s. f.). *CMake — Reference Documentation*. Recuperado de <https://cmake.org/cmake/help/latest/>
- Microsoft. (s. f.). *NuGet — Documentation*. Recuperado de <https://learn.microsoft.com/en-us/nuget/>
- npm, Inc. (s. f.). *npm Docs*. Recuperado de <https://docs.npmjs.com/>
- Open Container Initiative. (s. f.). *OCI Image Specification*. Recuperado de <https://specs.opencontainers.org/image-spec/>
- pnpm. (s. f.). *pnpm — Documentation*. Recuperado de <https://pnpm.io/>
- PyPA. (s. f.). *pip — Documentation*. Recuperado de <https://pip.pypa.io/>
- Python Poetry. (s. f.). *Poetry — Documentation*. Recuperado de <https://python-poetry.org/docs/>
- Reproducible Builds. (s. f.). *Reproducible Builds — Project Site*. Recuperado de <https://reproducible-builds.org/>
- Rust Project Developers. (s. f.). *The Cargo Book*. Recuperado de <https://doc.rust-lang.org/cargo/>
- Semantic Versioning. (s. f.). *Semantic Versioning 2.0.0*. Recuperado de <https://semver.org/>
- Sigstore Project. (s. f.). *Cosign — Documentation*. Recuperado de <https://docs.sigstore.dev/cosign/>
- SLSA Community. (s. f.). *Supply-chain Levels for Software Artifacts (SLSA)*. Recuperado de <https://slsa.dev/>
- SPDX Project (Linux Foundation). (s. f.). *SPDX — Sitio oficial*. Recuperado de <https://spdx.dev/>
- SPDX Project (Linux Foundation). (s. f.). *SPDX License List*. Recuperado de <https://spdx.org/licenses/>
- Yarn Project. (s. f.). *Yarn — Documentation*. Recuperado de <https://yarnpkg.com/>